Convergence Under Constraint

A refusal-capable framework for LLM problem-solving that separates the possible from the achievable

Kunal Das

Working paper · July 2026

ABSTRACT

Large language models are increasingly asked to *solve* problems, not merely to describe them. Yet the dominant failure mode of such systems is not the wrong answer — it is the *confident* wrong answer: a fluent, rigorous-looking solution to a question that was never answerable in the first place. This paper presents a convergence framework that treats problem-solving as a disciplined pipeline with an explicit formalization front-end and a refusal-capable core. The framework rests on three claims. First, that solving and verifying are asymmetric activities that must be architecturally separated. Second, that the translation of a natural-language goal into a formal objective function is both automatable and the single most dangerous step in the pipeline, because it is where the *formalizability wall* and the *specification–intent gap* live. Third, that mathematical *possibility* does not imply *achievability*: a goal may be well-posed and still be unreachable because of an information wall, a formalizability wall, or a complexity wall — and only some of these walls yield to a more capable engine. We argue that the framework's principal contribution is epistemic honesty: it is designed so that failures surface as explicit refusals rather than as silent, plausible errors. We close with a worked case study — a cryptocurrency trading system whose central defect is not a bug but a reality-limited objective — and a discussion of why the framework's efficacy is bounded above by the underlying model and below by nothing at all.

1. Introduction

The practical question that motivates this work is deceptively simple: *when we hand a goal to a language model, how do we know whether the goal can be reached at all?* Most tooling around LLMs optimizes for producing an answer. Very little optimizes for the prior question of whether an answer exists, whether it is constructible, and whether the model in front of us is the right instrument to construct it. The cost of skipping that prior question is not abstract. A system that always answers will, when handed an unanswerable goal, manufacture an answer — and it will do so with exactly the same fluency it brings to answerable ones. The user has no signal to distinguish the two.

We call the desired property *convergence under constraint*: the system should move toward a correct solution when one is reachable, and should *stop and say so* when one is not. This paper describes a framework — delivered in practice as a structured system prompt rather than as compiled software — that aims at this property. The framework is not an algorithm that guarantees correctness; no prompt can guarantee that. It is a *discipline* that channels whatever capability the underlying model has into a process whose failures are legible.

The remainder of the paper is organized around the framework's load-bearing ideas: the separation of verification from solution (§2), the formalization front-end that converts natural language into an objective function (§3), the distinction between possibility and achievability (§4), the three walls that make possible goals unreachable (§5), the engine-limited/reality-limited dichotomy that predicts where more capability helps (§6), the honesty property (§7), the framework's dependence on the underlying model (§8), and a case study of a reality-limited objective (§9).

2. The two-part architecture: verifier and solution provider

The framework decomposes any problem-solving act into two roles that are deliberately kept distinct. A *solution provider* proposes candidate solutions. A *verifier* decides whether a candidate actually satisfies the objective. The separation matters because these two activities are not symmetric in difficulty.

The asymmetry is familiar from complexity theory. For a large class of problems, checking a proposed solution is dramatically cheaper than producing one — the intuition behind the P versus NP question. A framework that conflates the two inherits the worst of both: it trusts a generator that may be wrong, with no independent check. By contrast, when verification is a separate, adversarial step, a weak solution provider can still yield a trustworthy system, because bad candidates are caught rather than shipped.

There is, however, a hazard unique to language models. When the verifier is *itself* a language model, it can hallucinate a passing verdict — it can declare a candidate correct because the candidate *reads* correct. A verifier that is not grounded in something outside the model's own prior — a test, a dataset, a proof rule, an oracle — is not really a verifier; it is a second opinion from the same mind. The framework therefore insists that verification be anchored to a ground truth wherever one exists, and that where none exists, the system say so rather than substitute confidence for evidence.

3. Phase F: from natural language to an objective function

A user does not arrive with an objective function. They arrive with a sentence. The framework's front-end — which we call *Phase F (Formalization)* — is the stage that converts the natural-language goal into the precise objective the rest of the pipeline requires, and then hands that objective forward for verification, solution, and achievement.

Phase F is automatable: a capable model drafts a reasonable objective function from a plain-language goal without difficulty. It is also, we argue, the *most dangerous step in the entire pipeline*. Two failure modes live here:

- **The specification–intent gap.** The framework verifies a solution against the *stated* objective, never against the user's unstated intent. If the formalization captures the wrong quantity, then a flawless downstream run produces a rigorous, correct, and useless answer — a precise answer to the wrong question. The danger is invisible precisely because everything after formalization is impeccable.

- **The formalizability wall.** Some goals cannot be reduced to a measurable objective at all — they are subjective, ill-posed, or lack any operational success criterion. The correct response is not to fabricate an objective; it is to stop and report that the goal is not formalizable, and to name what is missing.

Phase F is therefore built as a sequence of gates rather than a single translation:

Step	Function
F1 — Extract intent	Identify the decision variable(s), the meaning of "success," the direction (maximize / minimize / decide), the hard constraints, and the time horizon.
F2 — Draft candidate objective(s)	Write the objective explicitly. If several plausible formalizations exist, *enumerate* them rather than silently choosing one. The chosen objective determines the answer.
F3 — Label every input	Tag each input as descriptive, structural, or predictive. An input that merely describes the present, smuggled in as if it predicted the future, becomes a defect at the objective level.
F4 — Formalizability gate	If the goal cannot be reduced to a measurable objective, stop and return "not formalizable / underspecified," with the gap named. Do not invent an objective.
F5 — Intent verification	Restate the objective in plain language, with edge cases, and confirm it captures what the user meant. Mandatory, not optional.
F6 — Achievability pre-tag	Classify the goal as engine-limited or reality-limited, and note which wall (§5) it touches, so the user learns up front whether the solver can even in principle deliver.

Only a validated objective is handed forward. If F4 or F6 returns "impossible" or "not formalizable," that verdict is itself the deliverable — a valuable outcome, not a failure of the system.

4. Possibility is not achievability

A central discipline of the framework is to refuse to collapse two questions that ordinary language runs together. The first is whether a goal is *possible* — whether a solution exists in principle. The second is whether it is *achievable* — whether *we*, with our instrument and our data, can actually produce that solution. The framework restricts its attention to mathematically *possible* goals and then asks, within that class, a finer question.

Within the possible, we distinguish three grades:

- **Exists.** A solution provably exists, but no method to construct it is available. The canonical illustration is Zermelo's theorem: chess has a determined optimal outcome under perfect play, yet that solution is not in hand. Existence is proven; construction is not.
- **Constructible.** A solution can in principle be built by a known procedure, even if the procedure is impractical.
- **Tractable.** A solution can be built within the resources actually available — time, computation, data.

Confusing these grades is the source of a great deal of wasted effort. “It is possible” is often true and nearly always beside the point; the operative question is which of the three grades the goal occupies, and Phase F is where that determination is forced into the open.

One important corollary concerns the relationship between proving and making. For a *closed*, fully specified goal, a constructive proof that a solution exists can double as a recipe to build it. But this equivalence does not generalize. Definability does not imply provability, and provability does not imply constructibility — the lesson of Gödel's incompleteness results, Turing's halting problem, and Rice's theorem, each of which exhibits well-defined properties that no general procedure can decide. The framework treats “we can define it” as the beginning of the inquiry, never the end.

5. Three walls

When a possible goal turns out to be unachievable, the framework attributes the failure to one of three walls. Naming the wall is not academic: it determines whether a better engine could ever help (§6).

5.1 The empirical (information) wall

Some goals require predicting a target from inputs that simply do not contain the information needed. This wall is not a matter of cleverness; it is a matter of information content, and it is governed by a hard inequality. If a target `Y` is to be predicted from inputs `X`, then no transformation of those inputs — no model `g`, however sophisticated — can manufacture information about `Y` that the inputs did not already carry:

```
I( g(X) ; Y ) ≤ I( X ; Y )
```

This is the data-processing inequality. Its practical reading is stark: if the mutual information between your available inputs and the thing you want to predict is near zero, then no amount of modeling — and no increase in model capability — will produce a reliable prediction. The ceiling is set by the data, not by the engine. When a system's apparent signal is really a reflection of information already priced in by everyone else, the accessible forward information is what has already leaked away.

5.2 The formalizability wall

This is the wall Phase F's F4 gate exists to detect. A goal that admits no measurable success criterion cannot be optimized, because there is nothing to optimize *against*. Here the failure is upstream of any computation: you cannot solve what you cannot state. The honest output is a description of what would need to be pinned down before the problem even becomes a problem.

5.3 The complexity wall

Some goals are formalizable and information-rich yet remain out of reach because the computation required is infeasible — combinatorially explosive, or provably beyond available resources. The values of the Busy Beaver function and the digits of Chaitin's constant are extreme illustrations: perfectly well-defined, and yet uncomputable or unreachable in any practical sense. More mundanely, many optimization problems are tractable in the small and hopeless at scale.

6. Engine-limited versus reality-limited

The three walls sort into two categories, and this sorting is the framework's most actionable prediction. A goal is *engine-limited* if the obstacle is our current method or model — a more capable engine would break through. A goal is *reality-limited* if the obstacle is external to any engine — the information is not present, the objective is not formalizable, or the computation is provably infeasible — and no engine, however powerful, changes that.

Wall	Category	Does a stronger engine (up to ASI) help?
Complexity (method-bound)	Engine-limited	Often yes — better methods, heuristics, or scale can break through.
Empirical / information	Reality-limited	No — the ceiling is set by the data-processing inequality.
Formalizability	Reality-limited	No — there is nothing to optimize against.

The distinction cuts against a common intuition. It is tempting to believe that a sufficiently advanced intelligence — an AGI or ASI — could do anything. The framework's position is more precise: a superior engine dissolves engine-limited walls and leaves reality-limited walls exactly where they stand. Protein-structure prediction is a clean example of an engine-limited problem that yielded to a better engine: the information was present in the sequence, and a stronger method extracted it. A problem whose target carries no accessible forward information in its inputs is the opposite case; there is no method, present or future, that predicts what the data does not encode.

7. The honesty property

The framework's most important output is not a solution but a *verdict about solvability*. Its design goal is to convert the failure mode of language models from “confidently wrong” to “honestly limited.” Under this discipline, a system handed an unreachable goal is pushed to fail *loudly* — flagging a missing predictive input, refusing to formalize an ill-posed objective, naming the wall — rather than failing *silently* with a plausible answer.

> *A refusal that is correct is worth more than a solution that is confident and wrong. The former protects the user; the latter is a trap disguised as help.*

This reframes refusal as a first-class result. In most tools, “I cannot do this” is treated as a degenerate outcome. In this framework it is frequently the *most valuable* outcome, because it is the one the user cannot easily obtain elsewhere and the one that saves the most wasted effort.

8. Model dependence: what transfers and what scales

Because the framework is a prompt and not compiled code, a natural question is whether it works uniformly across models — whether the choice of engine (one general-purpose model versus another) matters. The answer separates cleanly into two parts.

What is model-agnostic. The *structure* transfers to any capable model: the gates, the input labels, the requirement to name a wall, the mandate to refuse when the objective is not formalizable. The honesty property — the conversion of silent error into loud limitation — is roughly universal in *shape*. This raises the floor on any model.

What scales with the model. Every actual judgment inside the framework is performed by the model, so quality tracks the model along several axes: instruction adherence (does it actually stop at the refusal gates, or answer anyway?), reasoning and verification depth (can it check as well as generate?), calibration and resistance to sycophancy (does it manufacture a passing verdict to please?), context capacity (can it hold a large artifact in view?), and stability (does it converge or thrash?). Of these, instruction adherence is the dominant source of variance: the entire discipline rests on the model obeying "stop and refuse" when the gates so require.

The framework channels capability; it does not create it. It cannot make a non-reasoning model reason, and it cannot conjure information the data lacks. On a frontier reasoning model, one approaches the framework's full value; on a weaker model, one still obtains the honesty benefit — and, crucially, obtains it *knowing* the solution is weaker, because the framework forces the model to declare its walls. "Deterministic" in this context refers to the procedure, not the output: the same gates run every time, but a stochastic model may still return different results on different runs.

9. Case study: a reality-limited objective

The framework was stress-tested against a concrete, adversarial domain: a short-horizon cryptocurrency trading system whose objective was to maximize expected net profit per trade after all costs. The system was engineered carefully — scoring, liquidity gates, cost and slippage models, an expected-value margin gate. Nine distinct defects were identified and repaired. And yet the framework's verdict on the central goal was negative, for a reason that no amount of further engineering could address.

The decisive defect — catalogued as the objective-level defect — was not a bug. It was that the system had *no validated forward-predictive input*. Its scoring reconstructed a signal from lagging, public trade statistics — buy/sell counts and volume — quantities that describe what the herd has *already* done. A composite of such descriptors is a description of the present, not a prediction of the future. Labeled honestly under Phase F's F3 step, these are descriptive inputs masquerading as predictive ones.

This is the empirical wall of §5.1 in concrete form. The target — future price movement — is being predicted from inputs whose accessible forward information about that target is close to nil, because whatever information they carried has already been acted upon by other participants. By the data-processing inequality, no scoring function built on those inputs can exceed that ceiling. The correct verdict is therefore not "fix the score" but "this objective is reality-limited given these inputs." The defect reopens only if a genuinely new, causally upstream, forward-predictive input can be found and validated out of sample — a substantive empirical requirement, not a coding change.

The case study also illustrates a methodological trap the framework is designed to resist: the temptation to run a retrospective backtest and read a passing number as vindication. A backtest

whose own scoring omits the very component that would carry predictive signal cannot resolve the objective-level defect; a “pass” from such a test is false comfort. The framework's discipline is to refuse to let a favorable-looking runtime result overrule a structural verdict about information. The valuable output was the honest conclusion: keep the framework, which did its job, and abandon the belief that this particular objective was achievable with these inputs.

10. Discussion and limitations

The framework is a discipline, not a proof system, and it inherits the limits of the model that runs it (§8). It cannot guarantee that a formalization is faithful — F5 mitigates but cannot eliminate the specification–intent gap, because intent is not fully observable. It cannot guarantee that a verifier is grounded when no ground truth exists; in that regime it can only insist on honesty about the absence. And because it is expressed as natural-language instruction, its enforcement is probabilistic: a model that does not follow instructions will not follow these.

What the framework offers, in return for these limits, is a reliable change in the *distribution* of failures. It does not make failure impossible; it makes failure legible. For a practitioner, the difference between a silent wrong answer and a loud, well-located refusal is the difference between a trap and a signpost.

11. Conclusion

We have described a framework whose organizing conviction is that the most useful thing an AI problem-solver can do is often to tell the truth about what cannot be done. By separating verification from solution, by making formalization an explicit and refusal-capable front-end, by refusing to conflate possibility with achievability, and by naming the wall whenever a goal proves unreachable, the framework converts the characteristic failure of language models — the confident, fluent, wrong answer — into an honest account of limits. Its ceiling rises with the capability of the engine that runs it; its floor — the guarantee that failures are declared rather than disguised — costs nothing and holds on any model that will follow instructions. In a landscape optimized to always answer, a system that knows when to stop is not a lesser tool. It is a more trustworthy one.

References and conceptual sources

1. Shannon, C. E. (1948). *A Mathematical Theory of Communication*. Bell System Technical Journal — foundation of mutual information.
2. Cover, T. M., & Thomas, J. A. (2006). *Elements of Information Theory*. Wiley — the data-processing inequality, $I(g(X);Y) \le I(X;Y)$.
3. Cook, S. (1971). *The Complexity of Theorem-Proving Procedures*. — origin of the P vs NP question and verify/solve asymmetry.
4. Zermelo, E. (1913). On an application of set theory to the theory of the game of chess — existence without construction.
5. Gödel, K. (1931). On formally undecidable propositions — definability does not imply provability.
6. Turing, A. M. (1936). On computable numbers — the halting problem.

7. Rice, H. G. (1953). Classes of recursively enumerable sets and their decision problems — undecidability of non-trivial semantic properties.
8. Radó, T. (1962). On non-computable functions — the Busy Beaver function.
9. Chaitin, G. J. (1975). A theory of program size — the halting probability Ω.
10. Grossman, S. J., & Stiglitz, J. E. (1980). On the impossibility of informationally efficient markets. *American Economic Review*.
11. Jumper, J., et al. (2021). Highly accurate protein structure prediction with AlphaFold. *Nature* — an engine-limited wall broken by a better engine.

This working paper synthesizes the Convergence Framework and its Phase F formalization front-end. Framework artifacts and the deterministic system-prompt variants are maintained separately.